Implement Custom Hooks

Problem: Build a small library of reusable React custom hooks that interviewers ask for most often: `useToggle`, `usePrevious`, `useDebounce`, `useThrottle`, `useLocalStorage`, `useAsync` / `useFetch`, `useEventListener`, `useOnClickOutside`, `useInterval` and `useMediaQuery`, plus a demo page that proves each one works.

Tech stack: React (Vite, JavaScript, Tailwind CSS v4)

Time limit: 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what it must do, what is out of scope. Write the assumptions down.
2. **List the interactions**: every click / key / timer and what should happen. This is the checklist.
3. **Sketch the architecture**: modules and components, one job each.
4. **Design the state**: the minimum source of truth, and who owns it. Everything else is derived.
5. **Define contracts**: data shapes, function signatures, API mocks.
6. **Trace one flow end to end** before writing code.
7. **Find the hard parts** early and decide how to solve them.
8. **Build in layers**: smallest working version first, then add one working layer at a time.
9. **Cover states and edge cases**: empty, loading, error, rapid input, unmount mid-request.
10. **Wrap up**: explain trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Use cases + hook signatures (contracts)	5 min
Simple hooks: `useToggle`, `useDebounce`, `usePrevious`	8 min
`useAsync` / `useFetch` with abort + race safety	12 min
`useLocalStorage` with cross-tab sync	8 min
`useLatest`, `useEventListener`, `useOnClickOutside`, `useInterval`	10 min
`useThrottle`, `useMediaQuery`	5 min
Demo page, edge cases, wrap-up	7 min

Think out loud

Say every decision before you type it: *"I'll return a tuple from `useToggle` like `useState` does, so callers can rename it; for `useFetch` I'll return an object because there are four fields."* The interviewer grades how you reason about **stale closures, cleanup and race conditions** more than how many hooks you finish. If time runs out, a correct `useDebounce` + `useFetch` beats ten buggy hooks.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- `useToggle(initial)` → `[on, toggle, setOn]`.
- `usePrevious(value)` → the value from the previous distinct render.
- `useDebounce(value, delay)` → value that updates only after `delay` ms of no changes.
- `useThrottle(value, interval)` → value that updates at most once per `interval`, and always settles on the last value.
- `useLocalStorage(key, initial)` → `[value, setValue]` like `useState`, persisted, survives reload, synced across tabs.
- `useAsync(asyncFn)` / `useFetch(url)` → `{ status, data, error, refetch }`, cancels stale requests.
- `useEventListener(type, handler, target)` → attach/detach a listener safely.
- `useOnClickOutside(ref, handler)` → call handler on pointer down outside the element.
- `useInterval(callback, delay)` → `setInterval` that can pause (`delay = null`) and never uses a stale callback.
- `useMediaQuery(query)` → boolean that updates when the media query result changes.

Non-functional requirements

Concern	What it means here
Performance	No re-subscribing listeners/timers on every render; stable returned functions (`useCallback`).
Correctness	No stale closures, no setState after unmount, no out-of-order responses.
Error handling	Corrupt `localStorage` JSON, blocked storage, HTTP errors, network errors, aborted requests.
Accessibility / usability	Demo uses labels, `aria-live` for async results, `aria-expanded` on the dropdown, Esc to close.
Reusability	Hooks know nothing about the demo; same API shape as React built-ins (`useState` -style tuples).
Extensibility	`useFetch` is a thin wrapper over `useAsync`; `useOnClickOutside` is built on `useEventListener`.

Clarifying questions to ask

1. Which hooks matter most to you: should I go deep on 3-4 (fetch, debounce, localStorage) or wide on 10?
2. Plain JavaScript or TypeScript? Any library allowed (React Query, lodash) or from scratch?
3. Does it need to be SSR-safe (Next.js), or browser only?
4. For `useFetch`: should it cache responses or dedupe identical requests, or just fetch + cancel?
5. Should `useLocalStorage` sync across tabs, and between two components in the same tab?
6. Which React version? (React 18/19 StrictMode double-runs effects, which affects how cleanup must be written.)

Assumptions we will build with

- React 19, JavaScript, browser only (SSR noted as a follow-up; `useMediaQuery` already has a server snapshot).
- From scratch, no libraries.
- `useFetch` fetches, aborts, retries manually; **no cache** (that is what React Query is for).
- `useLocalStorage` syncs across tabs via the `storage` event; same-tab sync between two components is out of scope.
- Values in `localStorage` are JSON.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type "a", "ar", "arj" quickly in search	Shows "Typing…"; only **one** request after 400 ms of silence
Search "a", then "zara" while "a" is still loading	"a" request is aborted; only Zara's result is shown
Search "fail"	Error message + Retry button; Retry re-runs the same request
Search "xyz"	Empty state: "No users match"
Clear the search box	Back to idle; no request sent
Debounced query changes from "kav" to "zar"	`usePrevious` shows previous: "kav"
`useFetch` with a good URL / broken URL	Success shows data; broken shows error; switching back recovers
Edit preferences, reload page	Values are restored from `localStorage`
Edit `localStorage` in another tab	This tab updates without reload
Corrupt JSON in storage	Falls back to the initial value, no crash
Open dropdown, pointer down outside	Closes
Open dropdown, press Esc	Closes
Pointer down on the trigger itself	Does not close-then-reopen
Start stopwatch, change speed to 10×	Speed changes immediately; interval is **not** restarted
Pause / Reset	Timer stops; Reset sets 0.0s

Action / input	Expected behavior / output
Move pointer fast over the box	Raw x updates every event; throttled x every ~300 ms and ends on the final x
Resize below 640px	"Mobile width: yes" without reload

This table is the implementation checklist and the test list.

3. Design the architecture

Module	Responsibility
`hooks/useLatest`	Ref that always holds the newest value (internal helper for "latest callback").
`hooks/useToggle`	Boolean state + stable `toggle`.
`hooks/usePrevious`	Remember the previous distinct value.
`hooks/useDebounce`	Delay a value until input settles.
`hooks/useThrottle`	Rate-limit a value.
`hooks/useLocalStorage`	`useState` + persistence + cross-tab sync.
`hooks/useAsync`	Generic async state machine with abort + refetch.
`hooks/useFetch`	`useAsync` specialised for `fetch(url)` JSON.
`hooks/useEventListener`	Subscribe to DOM events with latest handler.
`hooks/useOnClickOutside`	`useEventListener('pointerdown', …, document)` + `contains` check.
`hooks/useInterval`	Declarative `setInterval`.
`hooks/useMediaQuery`	`matchMedia` through `useSyncExternalStore`.
`api/users`	Mock search API with random latency and abort support.
`demos/*`	One card per hook group, proves behavior.

How they relate

```
App
├── SearchDemo      → useDebounce, usePrevious, useAsync → api/searchUsers
├── FetchDemo       → useFetch → useAsync
├── PreferencesDemo → useLocalStorage, useToggle
├── DropdownDemo    → useOnClickOutside → useEventListener → useLatest
│                     useEventListener (Esc), useToggle
├── StopwatchDemo   → useInterval → useLatest, useToggle
└── ViewportDemo    → useMediaQuery, useThrottle, useEventListener
```

Start simple: each hook is one file with one job. Only two "layering" decisions: `useFetch` reuses `useAsync`, and the listener/timer hooks reuse `useLatest`. Both came from seeing the same code twice, not from planning ahead.

4. Design the state / data model

Each hook owns its own small state. There is no global store.

Hook	State or field	Purpose
`useToggle`	`on`	The boolean.
`usePrevious`	`current`, `previous`	Detect a change during render; keep the old one.
`useDebounce`	`debounced`	The settled value.
`useThrottle`	`throttled`, ref `lastEmit`	Emitted value; timestamp of last emit (ref: no re-render needed).
`useLocalStorage`	`value`, `prevKey`	Current value; detect key change during render.
`useAsync`	`{ status, data, error }`, `attempt`	Async state machine; `attempt` bump = refetch.
`useLatest`	ref	Newest callback, read inside long-lived listeners.
`useMediaQuery`	none (external store)	`matchMedia` *is* the source of truth.

```
// useAsync state machine
const IDLE = { status: 'idle', data: undefined, error: null };
// 'idle' -> 'loading' -> 'success' | 'error' ; refetch -> 'loading' again
const [state, setState] = useState(IDLE);
const [attempt, setAttempt] = useState(0);
```

Senior note

- **Derive, do not store:** `isLoading` is `status === 'loading'`; "Typing…" is `query !== debounced`. One `status` string instead of `isLoading/isError/isSuccess` booleans makes impossible states (loading *and* error) impossible.
- **Ref vs state:** use state only for what the UI shows. Timestamps (`lastEmit`) and the latest callback live in refs because changing them must not re-render.
- **Who owns state:** the hook owns its internal state; the caller owns the *inputs* (`query`, `url`, `delay`). `useAsync` takes a memoized function, so the caller decides when a new request is needed.
- **External stores** (`matchMedia`, `localStorage`) are the real source of truth; React state is a mirror, kept in sync by subscriptions.

5. Design the data structures and contracts

Hook signatures (the public API)

Hook	Input	Output
`useToggle(initial = false)`	boolean	`[on, toggle, setOn]`
`usePrevious(value)`	any	previous distinct value or `undefined`

Hook	Input	Output
`useDebounce(value, delay = 300)`	any, ms	debounced value
`useThrottle(value, interval = 200)`	any, ms	throttled value
`useLocalStorage(key, initial)`	string, value or `() => value`	`[value, setValue]` (setValue accepts updater fn)
`useAsync(asyncFn | null)`	`(signal) => Promise` (memoized)	`{ status, data, error, refetch }`
`useFetch(url | null)`	string	same as `useAsync`
`useEventListener(type, handler, target = window)`	event, fn, window / document / element / ref	nothing
`useOnClickOutside(ref, handler)`	ref, fn	nothing
`useInterval(callback, delay | null)`	fn, ms or null (paused)	nothing
`useMediaQuery(query)`	CSS media query	boolean

Why tuples for some and objects for others: tuples (like `useState`) when there are 2-3 values the caller will rename; an object when there are 4+ named fields (`useAsync`).

Mock API contract

```
// searchUsers(query: string, signal?: AbortSignal) => Promise<User[]>
// User = { id: string, name: string }   // stable ids -> stable React keys
searchUsers('arj', signal);
// resolves after 300-1200 ms (random, so responses can arrive out of order):
[{ id: 'u3', name: 'Arjun Mehta' }]
// query containing "fail" rejects: Error('Server error (500). Please retry.')
// signal.abort() rejects: DOMException('Aborted', 'AbortError')
```

The random latency is deliberate: it makes the race condition reproducible in the demo.

Stored value contract (`localStorage['mc57:prefs']`)

```
{ "name": "Ashu", "compact": true }
```

Prefix the key (`mc57:`) so different apps on the same origin do not collide.

6. Trace the data flow

Debounced search, user types "arj" quickly:

```
keystrokes "a", "ar", "arj"  (100 ms apart)
        |
setQuery()  -> SearchDemo re-renders, input shows "arj", meta shows "Typing…"
        |
useDebounce: each change clears the previous setTimeout and starts a new 400 ms one
        |
400 ms of silence -> setDebounced("arj")
        |
useMemo creates a NEW search function because `debounced` changed
        |
useAsync effect sees a new asyncFn:
   1. cleanup of previous run -> controller.abort()  (old request ignored)
   2. new AbortController, status -> 'loading' (old data kept)
   3. asyncFn(signal) -> searchUsers("arj", signal)
        |
promise resolves -> signal not aborted? -> setState({ status: 'success', data })
        |
UI: list shows "Arjun Mehta"; usePrevious shows the last searched term
```

7. Important technical challenges

Challenge 1: Race conditions and setState after unmount in `useAsync`

Problem: user searches "a" (slow, 1.2 s) then "zara" (fast, 0.3 s). "zara" returns first, then "a" returns and **overwrites** it. The list now shows results for "a" while the box says "zara". Same bug if the component unmounts while a request is in flight.

Solution: one `AbortController` per run. The effect cleanup aborts it when a newer run starts or the component unmounts; every `setState` checks `signal.aborted` first. Abort also cancels the network request (saves bandwidth), and the `AbortError` is ignored by the same check.

```
useEffect(() => {
  if (!asyncFn) { setState(IDLE); return; }
  const controller = new AbortController();
  setState((s) => ({ ...s, status: 'loading', error: null }));
  asyncFn(controller.signal).then(
    (data) => { if (!controller.signal.aborted) setState({ status: 'success', data, error: null }); },
    (error) => { if (!controller.signal.aborted) setState((s) => ({ ...s, status: 'error', error }));
},
  );
  return () => controller.abort();
}, [asyncFn, attempt]);
```

`asyncFn` must be memoized by the caller (`useMemo` / `useCallback`), otherwise a new function every render means a new request every render. Passing `null` means "do nothing" (empty search box).

Challenge 2: Stale closures in long-lived listeners and timers

Problem: `setInterval(() => setElapsed(e => e + 100 * speed), 100)` captures `speed` from the first render. Changing speed to 10× does nothing. Adding `callback` to the effect deps fixes it but **restarts the interval on every render** (an inline arrow is new each render), so the timer drifts or never fires.

Solution: keep the latest callback in a ref (`useLatest`), and let the timer call `ref.current()`. The effect depends only on `delay`, so the interval is created once and always runs the newest callback. The same helper powers `useEventListener` and `useOnClickOutside`.

```
export function useLatest(value) {
  const ref = useRef(value);
  useLayoutEffect(() => { ref.current = value; }); // after render, before other effects
  return ref;
}

export function useInterval(callback, delay) {
  const callbackRef = useLatest(callback);
  useEffect(() => {
    if (delay == null) return;               // null = paused
    const id = setInterval(() => callbackRef.current(), delay);
    return () => clearInterval(id);
  }, [delay, callbackRef]);
}
```

Why update the ref in an effect, not during render: writing `ref.current = value` in render is a side effect; in concurrent rendering a discarded render could leave the ref pointing at a callback that never committed.

Challenge 3: `useLocalStorage` that never crashes and stays in sync

Problem: (a) `JSON.parse` on a corrupt value or `localStorage` access in Safari private mode throws and white-screens the app. (b) Reading storage on every render is slow. (c) Another tab changes the value and this tab shows stale data. (d) The `key` prop changes and the old value gets written under the new key.

Solution: (a) wrap read and write in `try/catch`, fall back to `initial`. (b) lazy initial state `useState(() => read(key, initial))` reads once. (c) listen to the `storage` event (it fires only in *other* tabs; `e.key === null` means `clear()`). (d) detect the key change during render and reload, the same pattern as `usePrevious`.

```
const [value, setValue] = useState(() => read(key, initial));
const [prevKey, setPrevKey] = useState(key);
if (prevKey !== key) { setPrevKey(key); setValue(read(key, initial)); }

useEffect(() => {
  try { window.localStorage.setItem(key, JSON.stringify(value)); } catch { /* quota */ }
}, [key, value]);

useEffect(() => {
  const onStorage = (e) => {
    if (e.storageArea === window.localStorage && (e.key === key || e.key === null)) setValue(read(key,
initial));
  };
  window.addEventListener('storage', onStorage);
  return () => window.removeEventListener('storage', onStorage);
}, [key]);
```

No ping-pong between tabs: writing the same string again does not fire another `storage` event.

Challenge 4: A throttle that does not drop the last value

Problem: a naive throttle ("ignore updates for 300 ms after an emit") drops the **final** pointer position: the user stops moving at x = 240 but the UI is stuck at x = 205.

Solution: leading + trailing throttle. Every change schedules an emit at `lastEmit + interval` (or now, if that is in the past). A newer change replaces the pending timer, but the target time stays the same, so it fires on a steady cadence and always with the newest value.

```
useEffect(() => {
  const wait = lastEmit.current + interval - Date.now();
  const id = setTimeout(() => { lastEmit.current = Date.now(); setThrottled(value); }, Math.max(0,
wait));
  return () => clearTimeout(id);
}, [value, interval]);
```

8. Implement in the correct order

#	Step	Working result
1	Vite + Tailwind scaffold, `App` with a grid of empty cards	Page renders
2	`useToggle` + `useDebounce`, wire the search input with a "Typing…" label	Debounce visible
3	Mock `searchUsers` with random delay + abort	Fake backend
4	`useAsync` with status machine (no abort yet), render idle/loading/success/empty	Search works
5	Add `AbortController` + `aborted` check + `refetch`; type "fail" to see error + Retry	Race-safe

#	Step	Working result
6	`useFetch` as a 10-line wrapper; FetchDemo with good/broken URL	Reuse proven
7	`usePrevious` (state pattern) in the search card	Previous term shown
8	`useLocalStorage`: lazy read + write effect, then try/catch, then `storage` event, then key change	Persists + syncs
9	`useLatest` + `useEventListener` + `useOnClickOutside`; Dropdown with Esc and outside click	Self-closing menu
10	`useInterval` with pause (`null`); Stopwatch with live speed change	No stale closure
11	`useThrottle` + `useMediaQuery`; Viewport card	Rate limiting + external store
12	Barrel `hooks/index.js`, a11y pass, edge-case pass from section 9	Done

9. Cover all states and edge cases

State / edge case	Handling
Initial (search empty)	`asyncFn = null` → status `idle`, "Start typing"
Typing (not settled)	`query !== debounced` → "Typing…", no request
Loading	`status === 'loading'`; old data kept so the list does not flash
Success	Render list with stable `id` keys
Empty result	`success` + `data.length === 0` → "No users match"
Error	`status === 'error'` → message + Retry (`refetch`)
Out-of-order responses	Older run aborted; its result ignored
Unmount mid-request	Cleanup aborts; no setState on unmounted component
HTTP 404/500	`fetch` resolves; we throw on `!res.ok`
SPA fallback returns HTML with 200	`res.json()` throws → error state (message shows the parse error)
StrictMode double effect	First run aborted by cleanup, second run wins; still one visible result
Corrupt / missing storage value	try/catch → `initial`
Storage blocked or quota full	Write fails silently; value still works in memory
`localStorage.clear()` in another tab	`e.key === null` → reload from storage → `initial`
Key changes	Reload value for the new key during render
Interval paused	`delay = null` → no interval
Callback changes every render	`useLatest`, timer not restarted
Pointer down on trigger	Inside the ref → not "outside"; trigger click toggles normally

State / edge case	Handling
Ref target not mounted yet	`useEventListener` skips (`el?.addEventListener` guard)
Media query changes (resize, OS theme)	`useSyncExternalStore` re-renders with new snapshot
SSR	`useMediaQuery` server snapshot `false`; other hooks touch `window` only in effects or lazy init (see wrap-up)

10. Final structure

```
src/
├── main.jsx
├── App.jsx
├── index.css
├── api/
│   └── users.js              # mock search API (random latency, abort, "fail" -> error)
├── hooks/
│   ├── index.js              # barrel export
│   ├── useLatest.js          # internal helper
│   ├── useToggle.js
│   ├── usePrevious.js
│   ├── useDebounce.js
│   ├── useThrottle.js
│   ├── useLocalStorage.js
│   ├── useAsync.js
│   ├── useFetch.js
│   ├── useEventListener.js
│   ├── useOnClickOutside.js
│   ├── useInterval.js
│   └── useMediaQuery.js
└── demos/
    ├── Card.jsx
    ├── SearchDemo.jsx
    ├── FetchDemo.jsx
    ├── PreferencesDemo.jsx
    ├── DropdownDemo.jsx
    ├── StopwatchDemo.jsx
    └── ViewportDemo.jsx
public/
└── quote.json                # target for useFetch demo
```

11. Full working code

Setup: `npm create vite@latest custom-hooks -- --template react`, then `npm i tailwindcss @tailwindcss/vite` and add `tailwindcss()` to the Vite plugins.

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/index.css

```
@import "tailwindcss";

body {
  background: #f8fafc;
  color: #0f172a;
}
```

src/hooks/useLatest.js

```
import { useLayoutEffect, useRef } from 'react';

// Ref that always holds the newest value, so long-lived listeners/timers never read a stale closure.
export function useLatest(value) {
  const ref = useRef(value);
  useLayoutEffect(() => {
    ref.current = value;
  });
  return ref;
}
```

src/hooks/useToggle.js

```
import { useCallback, useState } from 'react';

export function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn((v) => !v), []);
  return [on, toggle, setOn];
}
```

src/hooks/usePrevious.js

```
import { useState } from 'react';

// Previous *distinct* value. State (not a ref read during render) keeps it safe in concurrent
rendering.
export function usePrevious(value) {
  const [current, setCurrent] = useState(value);
  const [previous, setPrevious] = useState(undefined);
  if (!Object.is(current, value)) {
    setPrevious(current);
    setCurrent(value);
  }
  return previous;
}
```

src/hooks/useDebounce.js

```
import { useEffect, useState } from 'react';

export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);

  return debounced;
}
```

src/hooks/useThrottle.js

```
import { useEffect, useRef, useState } from 'react';

// Leading + trailing throttle: emits at most once per `interval`, and always emits the last value.
export function useThrottle(value, interval = 200) {
  const [throttled, setThrottled] = useState(value);
  const lastEmit = useRef(0);

  useEffect(() => {
    const wait = lastEmit.current + interval - Date.now();
    const id = setTimeout(() => {
      lastEmit.current = Date.now();
      setThrottled(value);
    }, Math.max(0, wait));
    return () => clearTimeout(id);
  }, [value, interval]);

  return throttled;
}
```

src/hooks/useLocalStorage.js

```
import { useEffect, useState } from 'react';

function read(key, initial) {
  try {
    const raw = window.localStorage.getItem(key);
    if (raw !== null) return JSON.parse(raw);
  } catch {
    // blocked storage or corrupt JSON: fall back to the initial value
  }
  return typeof initial === 'function' ? initial() : initial;
}

export function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => read(key, initial));
  const [prevKey, setPrevKey] = useState(key);

  // Key changed: load the new key's value during render instead of writing the old value under it.
  if (prevKey !== key) {
    setPrevKey(key);
    setValue(read(key, initial));
  }

  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(value));
    } catch {
      // quota exceeded / private mode: keep working in memory
    }
  }, [key, value]);

  // Other tabs: the `storage` event fires only in tabs that did NOT make the change.
  useEffect(() => {
    const onStorage = (e) => {
      if (e.storageArea === window.localStorage && (e.key === key || e.key === null)) {
        setValue(read(key, initial));
      }
    };
    window.addEventListener('storage', onStorage);
    return () => window.removeEventListener('storage', onStorage);
    // `initial` is only a fallback; re-subscribing on every new object literal would be wasteful
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [key]);

  return [value, setValue];
}
```

src/hooks/useAsync.js

```
import { useCallback, useEffect, useState } from 'react';

const IDLE = { status: 'idle', data: undefined, error: null };

// Runs asyncFn(signal) whenever its identity changes. Pass a memoized function, or null to stay idle.
export function useAsync(asyncFn) {
  const [state, setState] = useState(IDLE);
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    if (!asyncFn) {
      setState(IDLE);
      return;
    }
    const controller = new AbortController();
    // keep old data while loading (stale-while-revalidate) so the UI does not flash empty
    setState((s) => ({ ...s, status: 'loading', error: null }));

    asyncFn(controller.signal).then(
      (data) => {
        if (!controller.signal.aborted) setState({ status: 'success', data, error: null });
      },
      (error) => {
        if (!controller.signal.aborted) setState((s) => ({ ...s, status: 'error', error }));
      },
    );
    // a newer run or unmount aborts this one, so a slow old response can never overwrite a newer one
    return () => controller.abort();
  }, [asyncFn, attempt]);

  const refetch = useCallback(() => setAttempt((n) => n + 1), []);
  return { ...state, refetch };
}
```

src/hooks/useFetch.js

```
import { useMemo } from 'react';
import { useAsync } from './useAsync';

export function useFetch(url) {
  const fetcher = useMemo(
    () =>
      url
        ? (signal) =>
            fetch(url, { signal }).then((res) => {
              // fetch only rejects on network failure; HTTP errors must be thrown by hand
              if (!res.ok) throw new Error(`HTTP ${res.status}`);
              return res.json();
            })
        : null,
    [url],
  );
  return useAsync(fetcher);
}
```

src/hooks/useEventListener.js

```
import { useEffect } from 'react';
import { useLatest } from './useLatest';

// target: window (default), document, an element, or a ref to an element.
export function useEventListener(type, handler, target = window) {
  const handlerRef = useLatest(handler);

  useEffect(() => {
    const el = target && 'current' in target ? target.current : target;
    if (!el?.addEventListener) return;
    const listener = (e) => handlerRef.current(e);
    el.addEventListener(type, listener);
    return () => el.removeEventListener(type, listener);
  }, [type, target, handlerRef]);
}
```

src/hooks/useOnClickOutside.js

```
import { useEventListener } from './useEventListener';

export function useOnClickOutside(ref, handler) {
  // pointerdown covers mouse + touch + pen, and fires before focus moves
  useEventListener(
    'pointerdown',
    (e) => {
      if (ref.current && !ref.current.contains(e.target)) handler(e);
    },
    document,
  );
}
```

src/hooks/useInterval.js

```
import { useEffect } from 'react';
import { useLatest } from './useLatest';

// delay = null pauses. The callback can change every render without restarting the timer.
export function useInterval(callback, delay) {
  const callbackRef = useLatest(callback);

  useEffect(() => {
    if (delay == null) return;
    const id = setInterval(() => callbackRef.current(), delay);
    return () => clearInterval(id);
  }, [delay, callbackRef]);
}
```

src/hooks/useMediaQuery.js

```
import { useCallback, useSyncExternalStore } from 'react';

export function useMediaQuery(query) {
  const subscribe = useCallback(
    (notify) => {
      const mql = window.matchMedia(query);
      mql.addEventListener('change', notify);
      return () => mql.removeEventListener('change', notify);
    },
    [query],
  );
  // server snapshot: no window on the server, assume "no match"
  return useSyncExternalStore(subscribe, () => window.matchMedia(query).matches, () => false);
}
```

src/hooks/index.js

```
export { useToggle } from './useToggle';
export { usePrevious } from './usePrevious';
export { useDebounce } from './useDebounce';
export { useThrottle } from './useThrottle';
export { useLocalStorage } from './useLocalStorage';
export { useAsync } from './useAsync';
export { useFetch } from './useFetch';
export { useEventListener } from './useEventListener';
export { useOnClickOutside } from './useOnClickOutside';
export { useInterval } from './useInterval';
export { useMediaQuery } from './useMediaQuery';
```

src/api/users.js

```
const USERS = [
  { id: 'u1', name: 'Aarav Sharma' },
  { id: 'u2', name: 'Ananya Iyer' },
  { id: 'u3', name: 'Arjun Mehta' },
  { id: 'u4', name: 'Diya Kapoor' },
  { id: 'u5', name: 'Ishaan Verma' },
  { id: 'u6', name: 'Kavya Nair' },
  { id: 'u7', name: 'Rohan Gupta' },
  { id: 'u8', name: 'Saanvi Reddy' },
  { id: 'u9', name: 'Vihaan Joshi' },
  { id: 'u10', name: 'Zara Khan' },
];

// Mock API: random latency (so responses can arrive out of order), honours AbortSignal,
// and fails for any query containing "fail" so the error state can be tested.
export function searchUsers(query, signal) {
  return new Promise((resolve, reject) => {
    const id = setTimeout(() => {
      const q = query.trim().toLowerCase();
      if (q.includes('fail')) reject(new Error('Server error (500). Please retry.'));
      else resolve(USERS.filter((u) => u.name.toLowerCase().includes(q)));
    }, 300 + Math.random() * 900);

    signal?.addEventListener('abort', () => {
      clearTimeout(id);
      reject(new DOMException('Aborted', 'AbortError'));
    });
  });
}
```

public/quote.json

```
{ "id": 1, "text": "The best code is the code never written.", "author": "Every senior engineer" }
```

src/demos/Card.jsx

```
export default function Card({ title, hooks, children }) {
  return (
    <section className="rounded-xl border border-slate-200 bg-white p-5 shadow-sm">
      <h2 className="text-lg font-semibold text-slate-900">{title}</h2>
      <p className="mb-4 mt-1 flex flex-wrap gap-1.5">
        {hooks.map((h) => (
          <code key={h} className="rounded bg-indigo-50 px-1.5 py-0.5 text-xs text-indigo-700">
            {h}
          </code>
        ))}
      </p>
      {children}
    </section>
  );
}
```

src/demos/SearchDemo.jsx

```
import { useMemo, useState } from 'react';
import { searchUsers } from '../api/users';
import { useAsync, useDebounce, usePrevious } from '../hooks';
import Card from './Card';

export default function SearchDemo() {
  const [query, setQuery] = useState('');
  const debounced = useDebounce(query.trim(), 400);
  const previous = usePrevious(debounced);

  const search = useMemo(
    () => (debounced ? (signal) => searchUsers(debounced, signal) : null),
    [debounced],
  );
  const { status, data, error, refetch } = useAsync(search);
  const typing = query.trim() !== debounced;

  return (
    <Card title="Debounced search" hooks={['useDebounce', 'useAsync', 'usePrevious']}>
      <label htmlFor="search" className="sr-only">
        Search users
      </label>
      <input
        id="search"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder='Search users (type "fail" for an error)'
        className="w-full rounded-lg border border-slate-300 px-3 py-2 outline-none focus:border-
indigo-500"
      />
      <p className="mt-2 text-xs text-slate-500" data-testid="search-meta">
        {typing ? 'Typing…' : `Searched: "${debounced}"`}
        {previous ? ` · previous: "${previous}"` : ''}
      </p>

      <div className="mt-3 min-h-24" aria-live="polite" data-testid="search-result">
        {status === 'idle' && <p className="text-sm text-slate-500">Start typing to search.</p>}
        {status === 'loading' && <p className="text-sm text-slate-500">Loading…</p>}
        {status === 'error' && (
          <p className="text-sm text-red-600">
            {error.message}{' '}
            <button onClick={refetch} className="font-medium underline">
              Retry
            </button>
          </p>
        )}
        {status === 'success' && data.length === 0 && (
```

```
          <p className="text-sm text-slate-500">No users match "{debounced}".</p>
        )}
        {status === 'success' && data.length > 0 && (
          <ul className="space-y-1">
            {data.map((u) => (
              <li key={u.id} className="rounded bg-slate-50 px-3 py-1.5 text-sm">
                {u.name}
              </li>
            ))}
          </ul>
        )}
      </div>
    </Card>
  );
}
```

src/demos/FetchDemo.jsx

```
import { useState } from 'react';
import { useFetch } from '../hooks';
import Card from './Card';

const URLS = { ok: '/quote.json', broken: '/does-not-exist.json' };

export default function FetchDemo() {
  const [which, setWhich] = useState('ok');
  const { status, data, error, refetch } = useFetch(URLS[which]);

  return (
    <Card title="Fetch with abort + retry" hooks={['useFetch']}>
      <div className="flex flex-wrap gap-2">
        {Object.keys(URLS).map((k) => (
          <button
            key={k}
            onClick={() => setWhich(k)}
            aria-pressed={which === k}
            className={`rounded-lg px-3 py-1.5 text-sm ${which === k ? 'bg-indigo-600 text-white' :
'bg-slate-100'}`}
          >
            {k} URL
          </button>
        ))}
        <button onClick={refetch} className="rounded-lg bg-slate-100 px-3 py-1.5 text-sm">
          Refetch
        </button>
      </div>
      <div className="mt-3 min-h-16 text-sm" aria-live="polite" data-testid="fetch-result">
        {status === 'loading' && <p className="text-slate-500">Loading…</p>}
        {status === 'error' && <p className="text-red-600">Failed: {error.message}</p>}
        {status === 'success' && (
          <blockquote className="border-l-4 border-indigo-300 pl-3">
            “{data.text}” <span className="text-slate-500">— {data.author}</span>
          </blockquote>
        )}
      </div>
    </Card>
  );
}
```

src/demos/PreferencesDemo.jsx

```
import { useLocalStorage, useToggle } from '../hooks';
import Card from './Card';

const DEFAULT_PREFS = { name: '', compact: false };

export default function PreferencesDemo() {
  const [prefs, setPrefs] = useLocalStorage('mc57:prefs', DEFAULT_PREFS);
  const [showRaw, toggleRaw] = useToggle(false);

  return (
    <Card title="Persisted preferences" hooks={['useLocalStorage', 'useToggle']}>
      <label className="block text-sm font-medium" htmlFor="pref-name">
        Display name
      </label>
      <input
        id="pref-name"
        value={prefs.name}
        onChange={(e) => setPrefs((p) => ({ ...p, name: e.target.value }))}
        className="mt-1 w-full rounded-lg border border-slate-300 px-3 py-2 outline-none focus:border-
indigo-500"
      />
      <label className="mt-3 flex items-center gap-2 text-sm">
        <input
          type="checkbox"
          checked={prefs.compact}
          onChange={(e) => setPrefs((p) => ({ ...p, compact: e.target.checked }))}
        />
        Compact mode
      </label>
      <div className="mt-3 flex gap-2">
        <button onClick={toggleRaw} className="rounded-lg bg-slate-100 px-3 py-1.5 text-sm">
          {showRaw ? 'Hide' : 'Show'} stored JSON
        </button>
        <button onClick={() => setPrefs(DEFAULT_PREFS)} className="rounded-lg bg-slate-100 px-3 py-1.5
text-sm">
          Reset
        </button>
      </div>
      {showRaw && (
        <pre className="mt-3 rounded bg-slate-900 p-3 text-xs text-slate-100" data-testid="prefs-raw">
          {JSON.stringify(prefs, null, 2)}
        </pre>
      )}
      <p className="mt-3 text-xs text-slate-500">Reload, or edit in a second tab: the value stays in
sync.</p>
    </Card>
```

```
  );
}
```

src/demos/DropdownDemo.jsx

```
import { useRef, useState } from 'react';
import { useEventListener, useOnClickOutside, useToggle } from '../hooks';
import Card from './Card';

const OPTIONS = ['Edit', 'Duplicate', 'Archive', 'Delete'];

export default function DropdownDemo() {
  const [open, toggle, setOpen] = useToggle(false);
  const [picked, setPicked] = useState(null);
  const ref = useRef(null);

  useOnClickOutside(ref, () => setOpen(false));
  useEventListener('keydown', (e) => {
    if (e.key === 'Escape') setOpen(false);
  });

  return (
    <Card title="Dropdown that closes itself" hooks={['useOnClickOutside', 'useEventListener',
'useToggle']}>
      <div ref={ref} className="relative inline-block">
        <button
          onClick={toggle}
          aria-haspopup="menu"
          aria-expanded={open}
          className="rounded-lg bg-indigo-600 px-4 py-2 text-sm font-medium text-white"
        >
          Actions ▾
        </button>
        {open && (
          <ul role="menu" className="absolute left-0 z-10 mt-1 w-40 rounded-lg border border-slate-200
bg-white py-1 shadow-lg">
            {OPTIONS.map((o) => (
              <li key={o} role="none">
                <button
                  role="menuitem"
                  onClick={() => {
                    setPicked(o);
                    setOpen(false);
                  }}
                  className="w-full px-3 py-1.5 text-left text-sm hover:bg-slate-100"
                >
                  {o}
                </button>
              </li>
            ))}
          </ul>
        )}
```

```
      </div>
      <p className="mt-3 text-sm text-slate-600" data-testid="dropdown-picked">
        {picked ? `Picked: ${picked}` : 'Click outside or press Esc to close.'}
      </p>
    </Card>
  );
}
```

src/demos/StopwatchDemo.jsx

```
import { useState } from 'react';
import { useInterval, useToggle } from '../hooks';
import Card from './Card';

const TICK = 100;

export default function StopwatchDemo() {
  const [running, toggleRunning, setRunning] = useToggle(false);
  const [elapsed, setElapsed] = useState(0);
  const [speed, setSpeed] = useState(1);

  // reads `speed` from this render's closure; useInterval picks up the new callback without resetting the timer
  useInterval(() => setElapsed((ms) => ms + TICK * speed), running ? TICK : null);

  return (
    <Card title="Stopwatch" hooks={['useInterval', 'useToggle']}>
      <p className="font-mono text-3xl tabular-nums" data-testid="stopwatch">
        {(elapsed / 1000).toFixed(1)}s
      </p>
      <div className="mt-3 flex flex-wrap items-center gap-2">
        <button onClick={toggleRunning} className="rounded-lg bg-indigo-600 px-3 py-1.5 text-sm text-white">
          {running ? 'Pause' : 'Start'}
        </button>
        <button
          onClick={() => {
            setRunning(false);
            setElapsed(0);
          }}
          className="rounded-lg bg-slate-100 px-3 py-1.5 text-sm"
        >
          Reset
        </button>
        <label className="text-sm">
          Speed{' '}
          <select value={speed} onChange={(e) => setSpeed(Number(e.target.value))} className="rounded border px-1">
            <option value={1}>1×</option>
            <option value={2}>2×</option>
            <option value={10}>10×</option>
          </select>
        </label>
      </div>
    </Card>
```

```
  );
}
```

src/demos/ViewportDemo.jsx

```
import { useRef, useState } from 'react';
import { useEventListener, useMediaQuery, useThrottle } from '../hooks';
import Card from './Card';

export default function ViewportDemo() {
  const isMobile = useMediaQuery('(max-width: 640px)');
  const prefersDark = useMediaQuery('(prefers-color-scheme: dark)');
  const [x, setX] = useState(0);
  const throttledX = useThrottle(x, 300);
  const box = useRef(null);

  useEventListener('pointermove', (e) => setX(Math.round(e.offsetX)), box);

  return (
    <Card title="Viewport & pointer" hooks={['useMediaQuery', 'useThrottle', 'useEventListener']}>
      <ul className="text-sm" data-testid="media">
        <li>Mobile width: {isMobile ? 'yes' : 'no'}</li>
        <li>Prefers dark: {prefersDark ? 'yes' : 'no'}</li>
      </ul>
      <div ref={box} className="mt-3 h-20 rounded-lg bg-gradient-to-r from-indigo-100 to-indigo-300"
data-testid="pointer-box" />
      <p className="mt-2 font-mono text-sm" data-testid="pointer">
        raw x: {x} · throttled x (300ms): {throttledX}
      </p>
    </Card>
  );
}
```

src/App.jsx

```
import DropdownDemo from './demos/DropdownDemo';
import FetchDemo from './demos/FetchDemo';
import PreferencesDemo from './demos/PreferencesDemo';
import SearchDemo from './demos/SearchDemo';
import StopwatchDemo from './demos/StopwatchDemo';
import ViewportDemo from './demos/ViewportDemo';

export default function App() {
  return (
    <main className="mx-auto max-w-5xl px-4 py-8">
      <h1 className="text-2xl font-bold text-slate-900">Custom Hooks</h1>
      <p className="mb-6 text-sm text-slate-600">Each card exercises one or more reusable hooks from
src/hooks.</p>
      <div className="grid gap-5 md:grid-cols-2">
        <SearchDemo />
        <FetchDemo />
        <PreferencesDemo />
        <DropdownDemo />
        <StopwatchDemo />
        <ViewportDemo />
      </div>
    </main>
  );
}
```

12. Wrap-up

Edge cases handled

- Out-of-order and aborted requests, unmount during a request, HTTP errors, manual retry.
- Corrupt / blocked / full `localStorage`, cross-tab changes, `clear()`, key change.
- Stale closures in intervals and event listeners without restarting them.
- Throttle always settles on the final value; debounce cancels on every change and on unmount.
- Outside-click uses `pointerdown` (mouse, touch, pen); Esc closes the menu.

Trade-offs to mention

- `useAsync` takes a **memoized function**, not a deps array. It is lint-friendly (`exhaustive-deps` checks the caller's `useMemo`), but forgetting to memoize causes a request loop. A deps-array API is easier to call but hides dependencies from the linter.
- **No cache / dedupe** in `useFetch`. Two components fetching the same URL make two requests. For real apps: React Query / SWR, or a small module-level `Map` cache.
- `usePrevious` returns the previous **distinct** value (state pattern, concurrent-safe). The classic ref version returns the value from the previous **render**, but reads a ref during render.
- `useLocalStorage` syncs across tabs but **not** between two hook instances in the same tab. Fix: a tiny store with `useSyncExternalStore` + a custom event.

- `useEventListener` with a ref target attaches on mount only; if the element mounts later, pass the element via a callback ref / state.
- Kept old `data` during `loading` (stale-while-revalidate). Some UIs prefer clearing it; that is one line.

What to add with more time

- Tests with React Testing Library `renderHook` + fake timers (`vi.useFakeTimers()`): debounce timing, throttle trailing emit, abort ordering, storage event.
- TypeScript generics: `useLocalStorage<T>`, `useAsync<T>` so `data` is typed.
- `useFetch` options: cache with TTL, retry with exponential backoff, `refetchOnWindowFocus`.
- SSR safety: guard `window` access in `useLocalStorage`'s lazy init, use `useSyncExternalStore` with a server snapshot.
- More hooks: `useIntersectionObserver` (infinite scroll), `useResizeObserver`, `useHover`, `useCopyToClipboard`, `useUndoRedo` (useReducer).

13. Counter questions the interviewer may ask

Requirements and API design

Q1. What makes something a "custom hook" and not just a function?

A function whose name starts with `use` and that calls other hooks. The prefix tells React's linter to enforce the rules of hooks (top level only, same order every render). If it calls no hooks, make it a plain utility function instead.

Q2. Why does `useToggle` return a tuple but `useAsync` an object?

Tuples are easy to rename (`const [open, toggleOpen] = useToggle()`), like `useState`. With four fields (`status, data, error, refetch`) positional order is error-prone, and callers usually want only some of them, so named fields win.

Q3. Do two components using the same hook share state?

No. Each call gets its own state; hooks share **logic**, not state. To share state, lift it up, use Context, or put it in an external store read with `useSyncExternalStore`.

Q4. Why not just use lodash `debounce` inside the component?

You can, but the debounced function must be created once (`useMemo` / `useRef`) and cancelled on unmount, and it must call the latest callback. `useDebounce(value)` sidesteps all that: it debounces a **value**, and React's effect cleanup cancels the timer for free.

State design

Q5. Why one `status` string instead of `isLoading`, `isError` booleans?

Booleans allow impossible combinations (`isLoading && isError`). A single status is a small state machine: `idle → loading → success | error`. Booleans are derived: `const isLoading = status === 'loading'`.

Q6. When do you use a ref instead of state inside a hook?

When the value must survive renders but changing it should **not** re-render: timer ids, last-emit timestamps, the latest callback, the AbortController. Anything the UI displays goes in state.

Q7. Your `usePrevious` uses two `useState` calls. Why not the classic `useRef` version?

The ref version writes `ref.current` in an effect and reads it during render; with concurrent rendering a discarded render can make it wrong, and it returns the previous *render's* value even if nothing changed. Setting state during render (React docs pattern) is safe and returns the previous *distinct* value. Mention both; pick based on which semantic the caller needs.

Challenge 1: async and races

Q8. What if the API is slow and the user types fast?

Debounce reduces requests to one per pause; abort cancels the in-flight one when a new query arrives. The UI shows "Typing…" then "Loading…" and keeps old results visible, so it never flashes empty.

Q9. Why check `signal.aborted` if abort already rejects the promise?

Not every async function honors the signal (a third-party SDK, a computation). The check guarantees a stale result is ignored even if the work was not actually cancelled. Abort is the optimisation; the check is the correctness guarantee.

Q10. In StrictMode your effect runs twice in dev. Is that a bug?

No, React mounts, unmounts and re-mounts to surface missing cleanup. Our cleanup aborts the first request, so only the second one sets state. If you see two requests in the Network tab in dev, that is expected; production runs once.

Q11. What if the caller forgets to memoize the function passed to `useAsync`?

A new function each render → the effect re-runs → a request → setState → re-render → infinite loop. Mitigations: document it, let `exhaustive-deps` flag the caller, or offer `useFetch(url)` which memoizes internally. Alternative design: accept a `key` (like React Query) and keep the function in a ref.

Q12. How would you add caching?

A module-level `Map<url, { data, time }>`. On run: if a fresh entry exists, return it immediately (status `success`), else fetch and store. Also store the in-flight promise so concurrent callers share one request (dedupe). Past that, use React Query.

Challenge 2: stale closures

Q13. Explain the stale closure bug in `setInterval`.

The interval callback is created once and closes over the state from that render. Later renders create new state values, but the interval still sees the old ones. Fix with functional updates (`setX(x => x + 1)`) when you only need the previous state, or a latest-ref (`useLatest`) when you need other props/state.

Q14. Why `useLayoutEffect` in `useLatest` and not `useEffect`?

Layout effects run before regular effects and before the browser paints, so any event or timer that fires afterwards already sees the new callback. With `useEffect` there is a tiny window where an event could call the previous callback. React's upcoming `useEffectEvent` solves exactly this problem.

Q15. Why does `useOnClickOutside` listen to `pointerdown` and not `click`?

`pointerdown` fires before focus moves and before the click completes, covers touch and pen, and avoids the bug where the same click that opens a menu (bubbling to `document`) immediately closes it.

Challenge 3: localStorage

Q16. Why does the `storage` event not fire in the tab that made the change?

By spec it fires only in *other* documents of the same origin. That is why we do not get a feedback loop. For same-tab sync between two hook instances, dispatch a custom event after `setItem` or use a shared store with `useSyncExternalStore`.

Q17. What if the stored data is 5 MB or the schema changes between releases?

`localStorage` is synchronous and limited (~5 MB); big data belongs in IndexedDB. For schema changes, store a version (`{ v: 2, data }`) and migrate or reset on read; our try/catch already prevents a crash on a bad shape.

Challenge 4 and performance

Q18. Debounce vs throttle: when do you use which?

Debounce when you care only about the **final** value after activity stops (search input, autosave, resize end). Throttle when you need **regular** updates during activity (scroll position, drag, pointer tracking, analytics).

Q19. What if 1,000 components each call `useEventListener('scroll')`?

1,000 listeners on `window` and 1,000 setStates per scroll frame. Better: one shared subscription (an external store that listens once and fans out via `useSyncExternalStore`), throttle with `requestAnimationFrame`, and `{ passive: true }` for scroll/touch listeners.

Q20. Why `useSyncExternalStore` for `useMediaQuery` instead of `useState` + `useEffect`?

With `useState` + `useEffect` the first render may show the wrong value, then flicker. `useSyncExternalStore` reads the real value during render, avoids tearing in concurrent rendering, and has a `getServerSnapshot` for SSR.

Trade-offs and testing

Q21. How do you test a custom hook?

`renderHook` from React Testing Library. Use fake timers for debounce/throttle/interval (`vi.advanceTimersByTime(400)`), resolve mocked promises in a controlled order to test races, and dispatch a `StorageEvent` for cross-tab sync.

```
const { result, rerender } = renderHook(({ v }) => useDebounce(v, 400), { initialProps: { v: 'a' } });
rerender({ v: 'ab' });
act(() => vi.advanceTimersByTime(399)); expect(result.current).toBe('a');
act(() => vi.advanceTimersByTime(1));   expect(result.current).toBe('ab');
```

Q22. When should you NOT write a custom hook?

When the logic does not use React state/effects (write a plain function), when it is used once and simple (inline it), or when a well-tested library already solves it in production (data fetching → React Query, forms → React Hook Form). Hooks are for reuse of **stateful** logic.